gp-research/ — Gaussian Processes for UUV Positioning

A self-contained knowledge base for using **Gaussian Processes (GPs)** to model the position of
an underwater unmanned vehicle (UUV/AUV) in this repository's navigation stack.

It is split in two:

- `**references/**` — teaching material. Concepts, math, intuition, and small worked examples.
Written to be generally educational; read these to understand before building.
- `**implementation/**` — build detail specific to this codebase (the DVL/IMU error-state EKF in
`dvl_correction/`). Read these when you are ready to design and code.

> Scope note: this round is documentation only. No source code, config, or dependencies are
> changed. Implementation happens in a later, separately-approved round, using
> `implementation/00-implementation-plan.md` as the spec.

Why GPs for an underwater vehicle?

Underwater there is **no GPS**. Position comes from dead-reckoning: integrate IMU acceleration and
DVL (Doppler Velocity Log) velocity over time. Integration means **error grows without bound**, and
the DVL — our best velocity sensor — **drops out** (loss of bottom-lock over soft/steep terrain,
aeration, steep banking) and occasionally returns **outliers**. A Gaussian Process is a natural fit
because it:

1. Models a quantity (velocity / position) **as a function of time or space**, with calibrated
uncertainty at every query point — exactly what a Kalman filter wants as a measurement.
2. **Interpolates and extrapolates** smoothly, so it can **bridge DVL gaps** with an uncertainty
that grows the longer the gap — graceful degradation, not a cliff.
3. Can be made to **respect physics** (ocean-current advection–diffusion) and boundary
conditions through the SPDE / second-order-PDE duality.
4. Has a **cheap, exact O(n) form** (the state-space / Kalman duality) suitable for the vehicle's
edge compute.

The three themes (and where they live)

Theme	What it answers	Primary docs
GPs for AUV navigation	How does a GP plug into DVL/IMU fusion and improve position?	`references/04`, `implementation/00–03`
The PDE / SPDE duality	How do we inject physical processes and boundary conditions?	`references/06`, `references/07`
Cheap edge compute	How do we run this on the vehicle without a GPU?	`references/05`, `references/08`, `implementation/04`

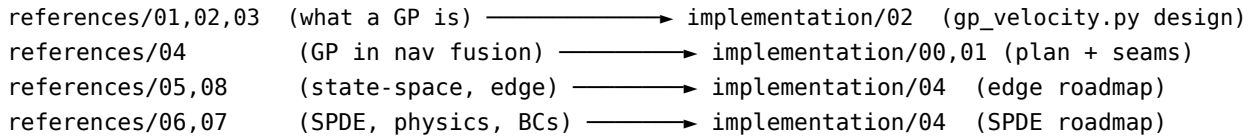
Suggested reading order

1. `references/00-overview-and-reading-map.md`
2. `references/01-gaussian-processes-fundamentals.md`
3. `references/02-kernels-and-covariance-functions.md`
4. `references/03-gp-regression-and-inference.md`
5. `references/04-gps-for-navigation-and-sensor-fusion.md`
6. `references/05-state-space-gps-and-the-kalman-duality.md`
7. `references/06-spde-approach-the-pde-duality.md`
8. `references/07-physics-informed-gps-and-boundary-conditions.md`
9. `references/08-cheap-edge-compute-for-gps.md`
10. `references/09-glossary-and-annotated-references.md`

Then the implementation set:

- `implementation/00-implementation-plan.md`
- `implementation/01-codebase-architecture-and-seams.md`
- `implementation/02-gp-velocity-module-design.md`
- `implementation/03-benchmark-and-frontend-demo.md`
- `implementation/04-edge-and-spde-roadmap.md`

How references/ maps to implementation/



Decisions already made (drive the implementation docs)

- First build: a **DVL-dropout velocity GP** (denoise + adaptive measurement noise + gap bridging).
- Posture: **demo-first, accuracy over optimization now**, with an **eventual on-vehicle edge** target.
- Tooling: **use a GP library** (GPyTorch) for the demo build; swap to the state-space form for edge.

See `references/09` for all sources.